

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники**

**Отчет по лабораторной работе №2
по дисциплине «Искусственный интеллект и машинное обучение»**

Выполнил студент Ромащенко Данил группы ИВТ-б-о-24-1

Подпись студента _____
Работа защищена « » _____ 20__ г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Тема: «Основы работы с библиотекой NumPy».

Цель работы: исследовать базовые возможности библиотеки NumPy языка программирования Python.

Ход работы

Вариант 17

Ссылка на репозиторий: https://github.com/I1N322/lab_2.git

1) Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирования Python.

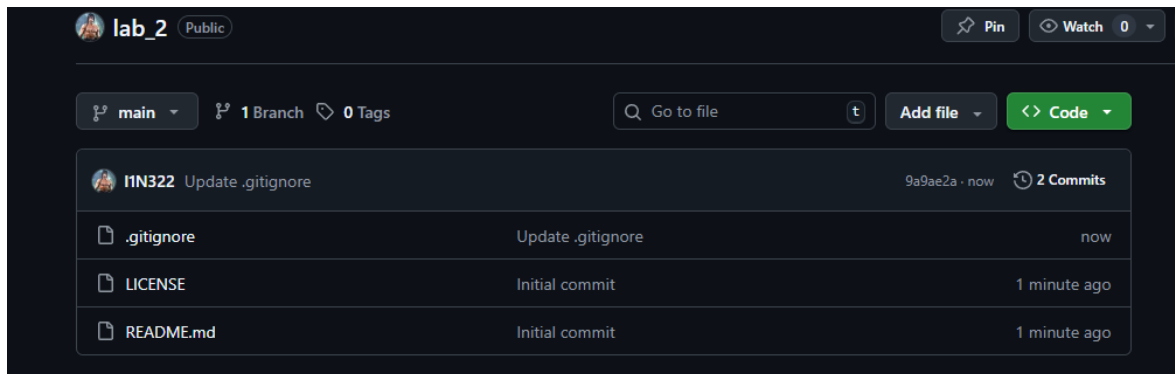


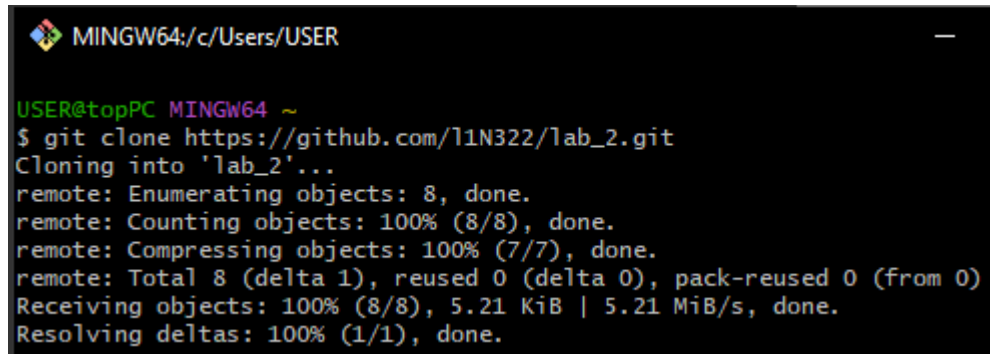
Рисунок 1. Созданный репозиторий

2) Был дополнен файл .gitignore необходимыми правилами.

```
209 # Covers JetBrains IDEs: IntelliJ, GoLand, RubyMine, PhpStorm, AppCode, PyCharm, CLion, Android Studio, WebStorm and Rider
210 # Reference: https://intellij-support.jetbrains.com/hc/en-us/articles/206544839
211
212 # User-specific stuff
213 .idea/**/workspace.xml
214 .idea/**/tasks.xml
215 .idea/**/usage.statistics.xml
216 .idea/**/dictionaries
217 .idea/**/shelf
218
219 # AWS User-specific
220 .idea/**/aws.xml
221
222 # Generated files
223 .idea/**/contentModel.xml
224
225 # Sensitive or high-churn files
226 .idea/**/dataSources/
227 .idea/**/dataSources.ids
228 .idea/**/dataSources.local.xml
229 .idea/**/sqlDataSources.xml
230 .idea/**/dynamic.xml
231 .idea/**/uiDesigner.xml
232 .idea/**/dbnavigator.xml
233
234 # Gradle
235 .idea/**/gradle.xml
236 .idea/**/libraries
237
238 # Gradle and Maven with auto-import
239 # When using Gradle or Maven with auto-import, you should exclude module files,
240 # since they will be recreated, and may cause churn. Uncomment if using
```

Рисунок 2. Добавленные правила в файл .gitignore

3) Было выполнено клонирование созданного репозитория на компьютер.



```
MINGW64:/c/Users/USER

USER@topPC MINGW64 ~
$ git clone https://github.com/l1N322/lab_2.git
Cloning into 'lab_2'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 5.21 KiB | 5.21 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

Рисунок 3. Клонирование репозитория

4) Были проработаны примеры.

```
[3]: import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = a + b
print(result)

[5 7 9]
```

Рисунок 4. Пример 1

```
[3]: import numpy as np
arr = np.array([1, 2, 3, 4])
print (arr)

[1 2 3 4]
```

Рисунок 5. Пример 2

```
[2]: import numpy as np
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])
result = np.dot(matrix1, matrix2)
print(result)

[[19 22]
 [43 50]]
```

Рисунок 6. Пример 3

```
[1]: import numpy as np
data = np.array([1, 2, 3, 4, 5])
mean = np.mean(data)
std_dev = np.std(data)
print(mean)
print(std_dev)

3.0
1.4142135623730951
```

Рисунок 7. Пример 4

```
[1]: import numpy as np
arr = np.array([1, 2, 3, 4])
result = arr * 2
print(result)
```

```
[2 4 6 8]
```

Рисунок 8. Пример 5

```
[2]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m)
```

```
[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
```

```
[3]: print(m[1, 0])
```

```
5
```

```
[4]: print(m[1, :])
```

```
[[5 6 7 8]]
```

```
[5]: print(m[:, 2])
```

```
[[3]
 [7]
 [5]]
```

```
[6]: print(m[1, 2:])
```

```
[[7 8]]
```

```
[7]: print(m[0:2, 1])
```

```
[[2]
 [6]]
```

```
[8]: print(m[0:2, 1:3])
```

```
[[2 3]
 [6 7]]
```

```
[10]: cols = [0, 1, 3]
print(m[:, cols])
```

```
[[1 2 4]
 [5 6 8]
 [9 1 7]]
```

```
[11]: print(type(m))
```

```
<class 'numpy.matrix'>
```

```
[12]: m = np.array(m)
print(type(m))
```

```
<class 'numpy.ndarray'>
```

```
[13]: print(m.shape)
```

```
(3, 4)
```

Рисунок 9. Пример 6

```
[2]: import numpy as np
      nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
      letters = np.array(['a', 'b', 'c', 'd', 'a', 'e', 'b'])

[3]: less_than_5 = nums < 5
      print(less_than_5)

      [ True  True  True  True False False False False False False]

[4]: pos_a = letters == 'a'
      print(pos_a)

      [ True False False False  True False False]

[5]: print(nums[less_than_5])

      [1 2 3 4]
```

Рисунок 10. Пример 7

```
[1]: import numpy as np
      m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
      print(m)

      [[1 2 3 4]
       [5 6 7 8]
       [9 1 5 7]]

[2]: mod_m = np.logical_and(m>=3, m<=7)
      print(mod_m)
      print(m[mod_m])

      [[False False  True  True]
       [ True  True  True False]
       [False False  True  True]]
      [[3 4 5 6 7 5 7]]

[3]: m[m > 7] = 25
      print(m)

      [[ 1  2  3  4]
       [ 5  6  7 25]
       [25  1  5  7]]
```

Рисунок 11. Пример 8

```
[1]: import numpy as np
      nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
      print(nums[nums < 5])

      [1 2 3 4]

[2]: nums[nums < 5] = 10
      print(nums)
```

[10 10 10 10 5 6 7 8 9 10]

Рисунок 12. Пример 9

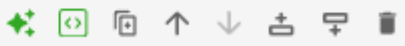
```
[1]: import numpy as np
print(np.arange(10))

[0 1 2 3 4 5 6 7 8 9]

[2]: print(np.arange(5, 12))

[ 5  6  7  8  9 10 11]

[3]: print(np.arange(1, 5, 0.5))
```



```
[1.  1.5 2.  2.5 3.  3.5 4.  4.5]
```

Рисунок 13. Пример 10

```
[1]: import numpy as np
a = [[1, 2], [3, 4]]
print(np.matrix(a))

[[1 2]
 [3 4]]

[2]: b = np.array([[5, 6], [7, 8]])
print(np.matrix(b))

[[5 6]
 [7 8]]

[5]: c = np.matrix('[1, 2; 3, 4]')
print(c)

[[1 2]
 [3 4]]
```

Рисунок 14. Пример 11

```
[3]: import numpy as np
print(np.zeros((3, 4)))

[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

[4]: print(np.eye(3))

[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Рисунок 15. Пример 12

```
[1]: import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(A)

[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
[2]: print(np.ravel(A))

[1 2 3 4 5 6 7 8 9]
```

```
[6]: print(np.ravel(A, order='C'))

[1 2 3 4 5 6 7 8 9]
```

```
[7]: print(np.ravel(A, order='F'))

[1 4 7 2 5 8 3 6 9]
```

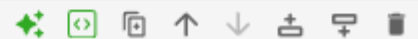


Рисунок 16. Пример 13

```
[2]: import numpy as np
a = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print(np.where(a % 2 == 0, a * 10, a / 10))

[ 0.  0.1 20.  0.3 40.  0.5 60.  0.7 80.  0.9]
```

```
[3]: b = np.random.rand(10)
print(b)
print(np.where(a > 0.5, True, False))
print(np.where(a > 0.5, 1, -1))

[0.34378268 0.55809065 0.7884672  0.96795295 0.67783739 0.01293337
 0.73523056 0.82164577 0.65331754 0.77433703]
[False True True True True True True True True True]
[-1  1  1  1  1  1  1  1  1  1]
```



Рисунок 17. Пример 14

```
[5]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

x = np.linspace(0, 1, 5)
print(x)
y = np.linspace(0, 2, 5)
print(y)
```

```
[0.  0.25 0.5  0.75 1.  ]
[0.  0.5 1.  1.5 2.  ]
```

```
[2]: xg, yg = np.meshgrid(x, y)
print(xg)
print(yg)
```

```
[[0.  0.25 0.5  0.75 1.  ]
 [0.  0.25 0.5  0.75 1.  ]
 [0.  0.25 0.5  0.75 1.  ]
 [0.  0.25 0.5  0.75 1.  ]
 [0.  0.25 0.5  0.75 1.  ]]
[[0.  0.  0.  0.  0.  ]
 [0.5 0.5 0.5 0.5 0.5]
 [1.  1.  1.  1.  1.  ]
 [1.5 1.5 1.5 1.5 1.5]
 [2.  2.  2.  2.  2.  ]]
```

```
[6]: plt.plot(xg, yg, color="r", marker="*", linestyle="none")
```

```
[6]: [<matplotlib.lines.Line2D at 0x1c5660e2210>,
<matplotlib.lines.Line2D at 0x1c5661da490>,
<matplotlib.lines.Line2D at 0x1c5661da5d0>,
<matplotlib.lines.Line2D at 0x1c5661da710>,
<matplotlib.lines.Line2D at 0x1c5661da850>]
```

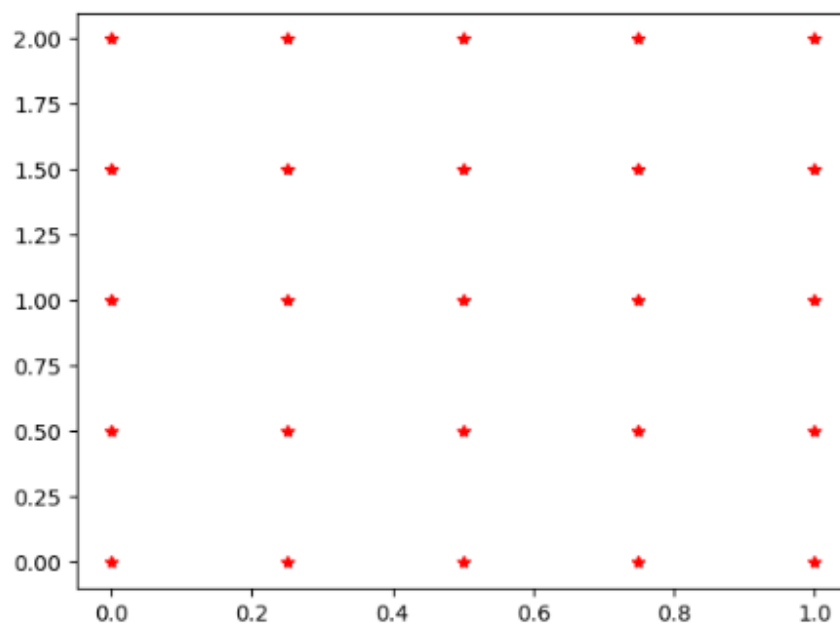


Рисунок 18. Пример 15


```
[1]: import numpy as np
print(np.random.permutation(7))
a = ['a', 'b', 'c', 'd', 'e']
print(np.random.permutation(a))

[4 5 3 0 2 1 6]
['e' 'c' 'b' 'd' 'a']

[2]: arr = np.linspace(0, 10, 5)
print(arr)

[ 0.  2.5  5.  7.5 10. ]

[3]: arr_mix = np.random.permutation(arr)
print(arr_mix)

[ 0. 10.  7.5  5.  2.5]

[5]: index_mix = np.random.permutation(len(arr_mix))
print(index_mix)
print(arr[index_mix])

[2 3 1 4 0]
[ 5.  7.5  2.5 10.  0. ]
```

Рисунок 19. Пример 16

```
[1]: import numpy as np
v_hor_np = np.array([1, 2])
print(v_hor_np)

[1 2]

[2]: v_hor_zeros_v1 = np.zeros((5, ))
print(v_hor_zeros_v1)

[0. 0. 0. 0. 0.]

[3]: v_hor_zeros_v2 = np.zeros((1, 5))
print(v_hor_zeros_v2)

[[0. 0. 0. 0. 0.]]

[4]: v_hor_one_v1 = np.ones((5, ))
print(v_hor_one_v1)
v_hor_one_v2 = np.ones((1, 5))
print(v_hor_one_v2)

[1. 1. 1. 1. 1.]
[[1. 1. 1. 1. 1.]]

[5]: v_vert_np = np.array([[1], [2]])
print(v_vert_np)

[[1]
 [2]]

[6]: v_vert_zeros = np.zeros((5, 1))
print(v_vert_zeros)

[[0.]
 [0.]
 [0.]
 [0.]
 [0.]]

[7]: v_vert_ones = np.ones((5, 1))
print(v_vert_ones)

[[1.]
 [1.]
 [1.]
 [1.]
 [1.]]
```

Рисунок 20. Пример 17

```
[1]: import numpy as np
m_sqr_arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m_sqr_arr)

[[1 2 3]
 [4 5 6]
 [7 8 9]]

[2]: m_sqr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
m_sqr_arr2 = np.array(m_sqr)
print(m_sqr_arr2)

[[1 2 3]
 [4 5 6]
 [7 8 9]]

[3]: m_sqr_mx = np.matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m_sqr_mx)

[[1 2 3]
 [4 5 6]
 [7 8 9]]

[4]: m_sqr_mx2 = np.matrix('1 2 3; 4 5 6; 7 8 9')
print(m_sqr_mx2)

[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Рисунок 21. Пример 18

```
[1]: import numpy as np
m_diag = [[1, 0, 0], [0, 5, 0], [0, 0, 9]]
m_diag_np = np.matrix(m_diag)
print(m_diag_np)

[[1 0 0]
 [0 5 0]
 [0 0 9]]

[2]: m_sqr_mx = np.matrix('1 2 3; 4 5 6; 7 8 9')
diag = np.diag(m_sqr_mx)
print(diag)

[1 5 9]

[3]: m_diag_np = np.diag(np.diag(m_sqr_mx))
print(m_diag_np)
```



```
[[1 0 0]
 [0 5 0]
 [0 0 9]]
```

Рисунок 22. Пример 19

```
[1]: import numpy as np
m_e = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
m_e_np = np.matrix(m_e)
print(m_e_np)

[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

```
[2]: m_eye = np.eye(3)
print(m_eye)

[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
[3]: m_zeros = np.zeros((3, 3))
print(m_zeros)

[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

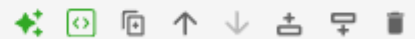


Рисунок 23. Пример 20

```
[2]: import numpy as np
m_mx = np.matrix('1 2 3; 4 5 6')
print(m_mx)

[[1 2 3]
 [4 5 6]]
```

```
[3]: m_var = np.zeros((2, 5))
print(m_var)

[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

Рисунок 24. Пример 21

```
[2]: import numpy as np
A = np.matrix('1 2 3; 4 5 6')
print(A)

[[1 2 3]
 [4 5 6]]
```

```
[3]: A_t = A.transpose()
print(A_t)

[[1 4]
 [2 5]
 [3 6]]
```

```
[4]: print(A.T)

[[1 4]
 [2 5]
 [3 6]]
```

```
[5]: R = (A.T).T
print(R)

[[1 2 3]
 [4 5 6]]
```

```
[7]: B = np.matrix('7 8 9; 0 7 5')
L = (A + B).T
R2 = A.T + B.T
print(L)
print(R2)

[[ 8  4]
 [10 12]
 [12 11]]
[[ 8  4]
 [10 12]
 [12 11]]
```

```
[9]: C = np.matrix('1 2; 3 4')
D = np.matrix('5 6; 7 8')
L2 = (C.dot(D)).T
R3 = (D.T).dot(C.T)
print(L2)
print(R3)

[[19 43]
 [22 50]]
[[19 43]
 [22 50]]
```

Рисунок 25. Пример 22

```
[1]: import numpy as np
A = np.matrix('1 2 3; 4 5 6')
C = 3 * A
print(C)
```

```
[[ 3  6  9]
 [12 15 18]]
```

```
[2]: B = np.matrix('1 2; 3 4')
L = 1 * B
R = B
print(L)
print(R)
```

```
[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
```

```
[4]: Z = np.matrix('0 0; 0 0')
L2 = 0 * B
R2 = Z
print(L2)
print(R2)
```

```
[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
```

```
[5]: p = 2
q = 3
L3 = (p + q) * B
R3 = p * B + q * B
print(L3)
print(R3)
```

```
[[ 5 10]
 [15 20]]
[[ 5 10]
 [15 20]]
```

```
[6]: L4 = (p * q) * B
R4 = p * (q * B)
print(L4)
print(R4)
```

```
[[ 6 12]
 [18 24]]
[[ 6 12]
 [18 24]]
```

Рисунок 26. Пример 23

```
[3]: import numpy as np
A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
C = A + B
print(C)
```

```
[[ 6  8]
 [10 12]]
```

```
[4]: C2 = B + A
print(C)
print(C2)
```

```
[[ 6  8]
 [10 12]]
[[ 6  8]
 [10 12]]
```

```
[5]: C = np.matrix('1 7; 9 3')
L = A + (B + C)
R = (A + B) + C
print(L)
print(R)
```

```
[[ 7 15]
 [19 15]]
[[ 7 15]
 [19 15]]
```

```
[6]: Z = np.matrix('0 0; 0 0')
L2 = A + (-1)*A
print(L2)
print(Z)
```

```
[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
```

Рисунок 27. Пример 24

```
[4]: import numpy as np
A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
C = np.matrix('2 4; 7 8')
L = A.dot(B.dot(C))
R = (A.dot(B)).dot(C)
print(L)
print(R)
```

```
[[192 252]
 [436 572]]
[[192 252]
 [436 572]]
```

```
[5]: L2 = A.dot(B + C)
R2 = A.dot(B) + A.dot(C)
print(L2)
print(R2)
```

```
[[35 42]
 [77 94]]
[[35 42]
 [77 94]]
```

```
[6]: L3 = A.dot(B)
R3 = B.dot(A)
print(L3)
print(R3)
```

```
[[19 22]
 [43 50]]
[[23 34]
 [31 46]]
```

```
[7]: E = np.matrix('1 0; 0 1')
I = E.dot(A)
Q = A.dot(E)
print(I)
print(Q)
```

```
[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
```

```
[8]: Z = np.matrix('0 0; 0 0')
I2 = Z.dot(A)
Q2 = A.dot(Z)
print(I2)
print(Q2)
```

```
[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
```

Рисунок 28. Пример 25

```
[1]: import numpy as np
A = np.matrix('-4 -1 2; 10 4 -1; 8 3 1')
print(A)

[[-4 -1  2]
 [10  4 -1]
 [ 8  3  1]]
```

```
[3]: print(np.linalg.det(A))

-14.000000000000009
```

```
[5]: print(A)
print(A.T)
det_A = round(np.linalg.det(A), 3)
det_A_t = round(np.linalg.det(A.T), 3)
print(det_A)
print(det_A_t)

[[-4 -1  2]
 [10  4 -1]
 [ 8  3  1]]
[[-4 10  8]
 [-1  4  3]
 [ 2 -1  1]]
-14.0
-14.0
```

```
[6]: B = np.matrix('-4 -1 2; 0 0 0; 8 3 1')
print(B)
print(np.linalg.det(B))

[[-4 -1  2]
 [ 0  0  0]
 [ 8  3  1]]
0.0
```

```
[7]: C = np.matrix('10 4 -1; -4 -1 2; 8 3 1')
print(A)
print(C)
print(round(np.linalg.det(A), 3))
print(round(np.linalg.det(C), 3))

[[-4 -1  2]
 [10  4 -1]
 [ 8  3  1]]
[[10  4 -1]
 [-4 -1  2]
 [ 8  3  1]]
-14.0
14.0
```

Рисунок 29. Пример 26


```
[1]: import numpy as np
A = np.matrix('-4 -1 2; 10 4 -1; 8 3 1')
k = 2
B = A.copy()
B[1, :] = B[1, :] + k * B[0, :]
print(A)
print(B)
print(round(np.linalg.det(A), 3))
print(round(np.linalg.det(B), 3))
```

```
[[ -4  -1   2]
 [ 10   4  -1]
 [  8   3   1]]
[[ -4  -1   2]
 [  2   2   3]
 [  8   3   1]]
-14.0
-14.0
```

```
[2]: print(A)
A[1, :] = A[0, :] + k * A[2, :]
print(round(np.linalg.det(A), 3))
```

```
[[ -4  -1   2]
 [ 10   4  -1]
 [  8   3   1]]
0.0
```

Рисунок 30. Пример 27

```

[1]: import numpy as np
      A = np.matrix('1 -3; 2 5')
      A_inv = np.linalg.inv(A)
      print(A_inv)

      [[ 0.45454545  0.27272727]
       [-0.18181818  0.09090909]]

[2]: A_inv_inv = np.linalg.inv(A_inv)
      print(A)
      print(A_inv_inv)

      [[ 1 -3]
       [ 2  5]]
      [[ 1. -3.]
       [ 2.  5.]]

[3]: L = np.linalg.inv(A.T)
      R = (np.linalg.inv(A)).T
      print(L)
      print(R)

      [[ 0.45454545 -0.18181818]
       [ 0.27272727  0.09090909]]
      [[ 0.45454545 -0.18181818]
       [ 0.27272727  0.09090909]]

[5]: B = np.matrix('7 6; 1 8')
      L2 = np.linalg.inv(A.dot(B))
      R2 = np.linalg.inv(B).dot(np.linalg.inv(A))
      print(L2)
      print(R2)

      [[ 0.09454545  0.03272727]
       [-0.03454545  0.00727273]]
      [[ 0.09454545  0.03272727]
       [-0.03454545  0.00727273]]

```

Рисунок 31. Пример 28

```
[1]: import numpy as np
m_eye = np.eye(4)
print(m_eye)

[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]

[2]: rank = np.linalg.matrix_rank(m_eye)
print(rank)

4

[3]: m_eye[3][3] = 0
print(m_eye)
rank = np.linalg.matrix_rank(m_eye)
print(rank)

[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  0.]]

3
```

Рисунок 32. Пример 29

5) Были выполнены задания по своему варианту (вариант 17).

Задание 1: Создайте массив NumPy размером 3×3 , содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

```
[3]: import numpy as np

arr = np.arange(1, 10).reshape(3,3)
arr *= 2
arr[arr > 10] = 0
print(arr)

[[ 2  4  6]
 [ 8 10  0]
 [ 0  0  0]]
```

Рисунок 33. Задание 1

6) Задание 2: Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

```
[4]: import numpy as np

arr = np.random.randint(1, 101, size=20)
print(f"Исходный массив: {arr}")
divide_by_5 = arr[arr % 5 == 0]
print(f"Элементы массива, которые делятся на 5 без остатка: {divide_by_5}")
arr[arr % 5 == 0] = -1
print(f"Обновленный массив: {arr}")
```

Исходный массив: [66 31 7 64 97 6 72 61 17 64 87 35 28 93 100 20 12 60
14 9]

Элементы массива, которые делятся на 5 без остатка: [35 100 20 60]

Обновленный массив: [66 31 7 64 97 6 72 61 17 64 87 -1 28 93 -1 -1 12 -1 14 9]

Рисунок 34. Задание 2

7) Задание 3: Создайте два массива NumPy размером 1×5 , заполненные случайными числами от 0 до 50. Объедините эти массивы в один двумерный массив (по строкам). Разделите полученный массив на два массива, каждый из которых содержит 5 элементов. Выведите все промежуточные и итоговые результаты.

```
[1]: import numpy as np

arr1 = np.random.randint(0, 51, size=(1, 5))
arr2 = np.random.randint(0, 51, size=(1, 5))

print("Первый массив:", arr1)
print("Второй массив:", arr2)

combined = np.vstack((arr1, arr2))
print("Объединённый двумерный массив:\n", combined)

split1, split2 = np.vsplit(combined, 2)
print("Первый разделённый массив:", split1)
print("Второй разделённый массив:", split2)
```

Первый массив: [[3 20 5 29 13]]

Второй массив: [[17 2 21 12 14]]

Объединённый двумерный массив:

```
[[ 3 20  5 29 13]
 [17  2 21 12 14]]
```

Первый разделённый массив: [[3 20 5 29 13]]

Второй разделённый массив: [[17 2 21 12 14]]

Рисунок 35. Задание 3

8) Задание 4: Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

```
[4]: import numpy as np

arr = np.linspace(-10, 10, 50)
print(arr)

arr_sum = np.sum(arr)
positive_sum = np.sum(arr[arr > 0])
negative_sum = np.sum(arr[arr < 0])

print("\nСумма всех элементов:", arr_sum)
print("Сумма положительных элементов:", positive_sum)
print("Сумма отрицательных элементов:", negative_sum)
```

```
[-10.          -9.59183673 -9.18367347 -8.7755102  -8.36734694
 -7.95918367 -7.55102041 -7.14285714 -6.73469388 -6.32653061
 -5.91836735 -5.51020408 -5.10204082 -4.69387755 -4.28571429
 -3.87755102 -3.46938776 -3.06122449 -2.65306122 -2.24489796
 -1.83673469 -1.42857143 -1.02040816 -0.6122449  -0.20408163
  0.20408163  0.6122449  1.02040816  1.42857143  1.83673469
  2.24489796  2.65306122  3.06122449  3.46938776  3.87755102
  4.28571429  4.69387755  5.10204082  5.51020408  5.91836735
  6.32653061  6.73469388  7.14285714  7.55102041  7.95918367
  8.36734694  8.7755102  9.18367347  9.59183673 10.         ]
```

```
Сумма всех элементов: 7.105427357601002e-15
Сумма положительных элементов: 127.55102040816328
Сумма отрицательных элементов: -127.55102040816327
```

Рисунок 36. Задание 4

9) Задание 5:

Создайте: Единичную матрицу размером 4×4. Диагональную матрицу размером 4×4 с диагональными элементами [5, 10, 15, 20] (не использовать циклы). Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

```
[1]: import numpy as np

idnt = np.eye(4)
diag = np.diag([5, 10, 15, 20])

sum_idnt = np.sum(idnt)
sum_diag = np.sum(diag)

print("Сумма элементов единичной матрицы:", sum_idnt)
print("Сумма элементов диагональной матрицы:", sum_diag)

if sum_idnt > sum_diag:
    print("Сумма элементов единичной матрицы больше")
elif sum_idnt < sum_diag:
    print("Сумма элементов диагональной матрицы больше")
else:
    print("Суммы равны")
```

```
Сумма элементов единичной матрицы: 4.0
Сумма элементов диагональной матрицы: 50
Сумма элементов диагональной матрицы больше
```

Рисунок 37. Задание 5

10) Задание 6: Создайте две квадратные матрицы NumPy размером 3×3 , заполненные случайными целыми числами от 1 до 20. Вычислите и выведите: Их сумму, их разность, их поэлементное произведение

```
[1]: import numpy as np

A1 = np.random.randint(1, 21, size=(3, 3))
A2 = np.random.randint(1, 21, size=(3, 3))

print("Матрица 1:\n", A1)
print("Матрица 2:\n", A2)

print("Сумма:\n", A1 + A2)
print("Разность:\n", A1 - A2)
print("Поэлементное произведение:\n", A1 * A2)
```

Матрица 1:

```
[[ 9  2  2]
 [ 6 14  6]
 [ 6  5 11]]
```

Матрица 2:

```
[[ 7  8 15]
 [ 1 17  7]
 [ 2  4  1]]
```

Сумма:

```
[[16 10 17]
 [ 7 31 13]
 [ 8  9 12]]
```

Разность:

```
[[ 2 -6 -13]
 [ 5 -3 -1]
 [ 4  1 10]]
```

Поэлементное произведение:

```
[[ 63 16 30]
 [ 6 238 42]
 [12 20 11]]
```

Рисунок 38. Задание 6

11) Задание 7:

Создайте две матрицы NumPy: Первую размером 2×3 , заполненную случайными числами от 1 до 10. Вторую размером 3×2 , заполненную случайными числами от 1 до 10. Выполните матричное умножение (@ или np.dot) и выведите результат.

```
[8]: import numpy as np

A = np.random.randint(1, 11, size=(2, 3))
B = np.random.randint(1, 11, size=(3, 2))
print("Первая матрица:\n", A)
print("Вторая матрица:\n", B)

result = np.dot(A, B)

print("Результат матричного умножения\n", result)

Первая матрица:
[[7 4 2]
 [4 1 4]]
Вторая матрица:
[[ 5  9]
 [ 8  8]
 [10  9]]
Результат матричного умножения
[[ 87 113]
 [ 68  80]]
```

Рисунок 39. Задание 7

12) Задание 8: Создайте случайную квадратную матрицу 3×3 . Найдите и выведите: Определитель этой матрицы, Обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена). Используйте функции `np.linalg.det` и `np.linalg.inv`.

```
[5]: import numpy as np

A = np.random.rand(3, 3)
print("Случайная матрица:\n", A)

A_det = round(np.linalg.det(A), 3)
print("\nОпределитель матрицы", A_det)

if A_det == 0:
    print("\nМатрица вырождена!")
else:
    A_inv = np.linalg.inv(A)
    print("\nОбратная матрица:\n", A_inv)

Случайная матрица:
[[0.60251541 0.97619503 0.99440634]
 [0.53962837 0.36543589 0.73280125]
 [0.53085885 0.35565058 0.27344496]]

Определитель матрицы 0.137

Обратная матрица:
[[-1.17447534  0.63385494  2.57242277]
 [ 1.76473512 -2.6540573   0.69495779]
 [-0.01517103  2.22139592 -2.24087339]]
```

Рисунок 40. Задание 8

13) Задание 9: Создайте матрицу NumPy размером 4×4, содержащую случайные целые числа от 1 до 50. Выведите: Исходную матрицу, Транспонированную матрицу, След матрицы (сумму элементов на главной диагонали). Используйте `np.trace` для нахождения следа.

```
[2]: import numpy as np

A = np.random.randint(1, 51, size = (4, 4))
A_t = np.transpose(A)
A_trace = np.trace(A)

print("Исходная матрица:\n", A)
print("Транспонированная матрица:\n", A_t)
print("След матрицы:\n", A_trace)

Исходная матрица:
[[42  7 28  7]
 [32 18 36 23]
 [42 11 39 28]
 [17 49 47 50]]
Транспонированная матрица:
[[42 32 42 17]
 [ 7 18 11 49]
 [28 36 39 47]
 [ 7 23 28 50]]
След матрицы:
149
```

Рисунок 41. Задание 9

14) Задание 10: Решите систему линейных уравнений вида:

$$2x + 3y - z = 5$$

$$4x - y + 2z = 6$$

$$-3x + 5y + 4z = -2$$

Используйте матричное представление $Ax = B$, где A - матрица коэффициентов, x - вектор неизвестных, B - вектор правой части. Решите систему с помощью `np.linalg.solve` и выведите результат.

```
[1]: import numpy as np

A = np.array([[2, 3, -1],
              [4, -1, 2],
              [-3, 5, 4]])

B = np.array([5, 6, -2])

x = np.linalg.solve(A, B)
print("Решение системы (x, y, z):", x)

Решение системы (x, y, z): [1.63963964 0.57657658 0.00900901]
```

Рисунок 42. Задание 10

15) Индивидуальное задание: Решите индивидуальное задание. Задание предусматривает построение системы линейных уравнений. Решите полученную систему уравнений с использованием библиотеки NumPy. Для решения системы используйте метод Крамера и матричный метод. Сравните полученные результаты, с результатами, полученными с помощью `np.linalg.solve`.

Распределение электроэнергии. Город получает электроэнергию от трех электростанций. Первая станция дает в два раза больше энергии, чем вторая, а третья - на 200 МВт больше, чем вторая. Всего требуется 1500 МВт. Сколько электроэнергии должно поступать с каждой станции?

Пусть x_1 , x_2 , x_3 - мощности (МВт) первой, второй, третьей станций соответственно.

$$x_1 = x_2 * 2$$

$$x_3 = x_2 + 200$$

$$x_1 + x_2 + x_3 = 1500$$

Получаем систему уравнений:

$$x_1 - x_2 * 2 = 0$$

$$x_3 - x_2 = 200$$

$$x_1 + x_2 + x_3 = 1500$$

```
[3]: import numpy as np

A = np.array([
    [1, -2, 0],
    [0, -1, 1],
    [1, 1, 1]
])
B = np.array([0, 200, 1500])

A_det = np.linalg.det(A)
if A_det == 0:
    print("Матрица вырождена, метод Крамера неприменим.")
else:
    x_cramer = []
    for i in range(3):
        Ai = A.copy()
        Ai[:, i] = B
        Ai_det = np.linalg.det(Ai)
        x_cramer.append(Ai_det / A_det)
    x_cramer = np.array(x_cramer)
    print("Метод Крамера:", x_cramer)

if A_det == 0:
    print("Матрица вырождена, обратная матрица не существует.")
else:
    A_inv = np.linalg.inv(A)
    x_inv = np.dot(A_inv, B)
    print("Матричный метод:", x_inv)

x_solve = np.linalg.solve(A, B)
print("np.linalg.solve:", x_solve)

print("\nВсе методы дают одинаковый результат (x1, x2, x3):")
print(f"x1 = {x_solve[0]}, x2 = {x_solve[1]}, x3 = {x_solve[2]}")

Метод Крамера: [650. 325. 525.]
Матричный метод: [650. 325. 525.]
np.linalg.solve: [650. 325. 525.]

Все методы дают одинаковый результат (x1, x2, x3):
x1 = 650.0, x2 = 325.0, x3 = 525.0
```

Рисунок 43. Индивидуальное задание

Ответы на контрольные вопросы:

1) NumPy (Numerical Python) — это фундаментальная библиотека для научных вычислений в Python. Она предоставляет:

- Многомерные массивы (ndarray) высокой производительности.
- Эффективные векторные и матричные операции.
- Широкий набор математических функций, работающих с массивами.
- Средства для линейной алгебры, преобразования Фурье, генерации случайных чисел и т.д.

– Основу для многих других библиотек (SciPy, pandas, scikit-learn, TensorFlow и др.).

2) Основная структура данных в NumPy - это многомерный массив, называемый ndarray. Это объект, который представляет собой таблицу данных (например, вектор или матрицу) и позволяет выполнять операции с ними с высокой эффективностью.

3) Доступ к частям многомерного массива осуществляется с помощью индексации и срезов.

4) NumPy предоставляет множество функций для расчёта статистических характеристик. Они могут применяться ко всему массиву или вдоль указанной оси.

Имя метода	Описание
argmax	Индексы элементов с максимальным значением (по осям)
argmin	Индексы элементов с минимальным значением (по осям)
max	Максимальные значения элементов (по осям)
min	Минимальные значения элементов (по осям)
mean	Средние значения элементов (по осям)
prod	Произведение всех элементов (по осям)
std	Стандартное отклонение (по осям)
sum	Сумма всех элементов (по осям)
var	Дисперсия (по осям)

5) Выборка — получение подмножества элементов. Способы: срезы, передача списков/массивов индексов, булева индексация.

6) Основные виды матриц и векторов:

– Вектор — одномерный массив (столбцовый или строчный, в NumPy различие не принципиально). `v = np.array([1,2,3])`

– Матрица — двумерный массив. `M = np.array([[1,2],[3,4]])`

– Квадратная матрица — число строк = число столбцов.

– Единичная матрица — единицы на главной диагонали. `I = np.eye(3)`

– Диагональная матрица — ненулевые элементы только на диагонали.
`D = np.diag([1,2,3])`

– Симметричная матрица — $A = A^T$.

7) Транспонирование — операция, при которой строки матрицы становятся столбцами, а столбцы — строками.

8) Свойства:

– Дважды транспонированная матрица равна исходной матрице

– Транспонирование суммы матриц равно сумме транспонированных матриц

– Транспонирование произведения матриц равно произведению транспонированных матриц расставленных в обратном порядке

– Транспонирование произведения матрицы на число равно произведению этого числа на транспонированную матрицу

– Определители исходной и транспонированной матрицы совпадают

9) Средства NumPy для транспонирования: атрибут `.T` — наиболее удобный; функция `np.transpose(a, axes=None)` — позволяет переставлять оси многомерного массива.

10) Основные действия над матрицами:

– Сложение и вычитание

– Умножение на число

– Умножение матриц (скалярное, матричное, поэлементное)

– Транспонирование

– Вычисление определителя (для квадратных)

– Нахождение обратной матрицы

11) При умножении матрицы на число, все элементы матрицы умножаются на это число.

12) Свойства:

– Произведение единицы и любой заданной матрицы равно заданной матрице.

– Произведение нуля и любой матрицы равно нулевой матрице, размерность которой равна исходной матрицы.

- Произведение матрицы на сумму чисел равно сумме произведений матрицы на каждое из этих чисел.

- Произведение матрицы на произведение двух чисел равно произведению второго числа и заданной матрицы, умноженному на первое число.

- Произведение суммы матриц на число равно сумме произведений этих матриц на заданное число.

13) Складывать (вычитать) можно только матрицы одинаковой размерности - то есть матрицы, у которых совпадает количество столбцов и строк.

14) Свойства:

- Коммутативность сложения. От перестановки матриц их сумма не изменяется.

- Ассоциативность сложения. Результат сложения трех и более матриц не зависит от порядка, в котором эта операция будет выполняться.

- Для любой матрицы существует противоположная ей, такая, что их сумма является нулевой матрицей.

15) Средства NumPy для сложения и вычитания: операторы `+` и `-`; функции `np.add()`, `np.subtract()`; `np.sum()` для суммирования всех элементов или по оси.

16) Умножение матриц это уже более сложная операция, по сравнению с рассмотренными выше. Умножать можно только матрицы, отвечающие следующему требованию: количество столбцов первой матрицы должно быть равно числу строк второй матрицы.

17) Свойства:

- Ассоциативность умножения. Результат умножения матриц не зависит от порядка, в котором будет выполняться эта операция.

- Дистрибутивность умножения. Произведение матрицы на сумму матриц равно сумме произведений матриц.

– Умножение матриц в общем виде не коммутативно. Это означает, что для матриц не выполняется правило независимости произведения от перестановки множителей.

– Произведение заданной матрицы на единичную равно исходной матрице.

– Произведение заданной матрицы на нулевую матрицу равно нулевой матрице.

18) Средства NumPy для умножения матриц: `@` — оператор (начиная с Python 3.5); `np.dot()` — также работает для скалярного произведения векторов.

19) Определитель матрицы размера (п-го порядка) является одной из ее численных характеристик. Определитель матрицы A обозначается как $|A|$ или $\det(A)$, его также называют детерминантом.

Свойства:

– Определитель матрицы остается неизменным при ее транспонировании.

– Если у матрицы есть строка или столбец, состоящие из нулей, то определитель такой матрицы равен нулю.

– При перестановке строк матрицы знак ее определителя меняется на противоположный.

– Если у матрицы есть две одинаковые строки, то ее определитель равен нулю.

– Если все элементы строки или столбца матрицы умножить на какое-то число, то и определитель будет умножен на это число.

– Если все элементы строки или столбца можно представить как сумму двух слагаемых, то определитель такой матрицы равен сумме определителей двух соответствующих матриц.

– Если к элементам одной строки прибавить элементы другой строки, умноженные на одно и то же число, то определитель матрицы не изменится.

– Если строка или столбец матрицы является линейной комбинацией других строк (столбцов), то определитель такой матрицы равен нулю.

– Если матрица содержит пропорциональные строки, то ее определитель равен нулю.

20) Средства NumPy для нахождения определителя: функция `np.linalg.det(A)`. Возвращает значение типа `float` (или комплексное).

21) Обратной матрицей A^{-1} матрицы A называют матрицу, удовлетворяющую следующему равенству: $A * A^{-1} = A^{-1} \times A = E$, где E - это единичная матрица.

Алгоритм нахождения:

- Вычислить определитель.
- Найти матрицу алгебраических дополнений.
- Транспонировать её — получить присоединённую матрицу.
- Разделить присоединённую матрицу на определитель.

22) Свойства:

- Обратная матрица обратной матрицы есть исходная матрица.
- Обратная матрица транспонированной матрицы равна транспонированной матрице от обратной матрицы.
- Обратная матрица произведения матриц равна произведению обратных матриц.

23) Функция `np.linalg.inv(A)` возвращает обратную матрицу, если она существует.

24) Каждая переменная выражается через определитель матрицы, где i -й столбец заменяется столбцом свободных членов. Формально это записывается как $x_i = \det(A_i) / \det(A)$, где A_i — матрица, полученная из A заменой i -го столбца на вектор b .

```
det_A = np.linalg.det(A)
```

```
if det_A == 0:
```

```
    print("Матрица вырождена, метод Крамера неприменим.")
```

```
else:
```

```
    x_cramer = []
```

```
    for i in range(3):
```

```
Ai = A.copy()
Ai[:, i] = B
det_Ai = np.linalg.det(Ai)
x_cramer.append(det_Ai / det_A)
x_cramer = np.array(x_cramer)
print("Метод Крамера:", x_cramer)
```

25) Матричный метод — это метод решения систем линейных уравнений, основанный на использовании обратной матрицы.

```
if det_A == 0:
    print("Матрица вырождена, обратная матрица не существует.")
else:
    A_inv = np.linalg.inv(A)
    x_inv = A_inv @ B
    print("Матричный метод:", x_inv)
```

Вывод: в ходе работы были исследованы базовые возможности библиотеки NumPy языка программирования Python.